

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

Національний аерокосмічний університет

«Харківський авіаційний інститут»



Факультет Інтелектуальних систем управління

Кафедра Систем управління літальними апаратами

Лабораторна робота № 9

з дисципліни «Алгоритмізація та програмування»

на тему: «Робота з рядками на C ++»

XAI.301.G12.311.8 ПР

Виконав: здобувач 1 курсу, групи 311
напряму підготовки (спеціальності) G12
Авіаційна та ракетно-космічна техніка
Максим ПАСНІЧЕНКО

Прийняла: доц. каф.301, канд. техн. наук,

_____ Олена ГАВРИЛЕНКО

МЕТА РОБОТИ

Вивчити теоретичний матеріал з основ роботи з низькорівневими рядками на C++ і документацію до класу `string`, а також алгоритми пошуку в рядку, а також реалізувати обробку рядків на C++ в середовищі QtCreator.

ПОСТАНОВКА ЗАДАЧІ

Завдання 1. (Варіант 18)

А. Вивчити по документації метод `replace` стандартного класу `string` відповідно до варіанту.

В. Визначити функцію `my_replace`, що виконує ті ж дії, що і вивчений метод класу `string`. Вихідний рядок передати першим параметром (масив символів). Для реалізації методу не використовувати функції обробки рядків зі стандартних бібліотек.

С. Викликати свій метод `my_replace` і метод `string::replace` аналогічно прикладам коду, наведеними в дод. А (через функцію `run_variant_18`). Перед викликом ввести з консолі один рядок і зберегти в масиві символів і змінній типу `string`.

Завдання 2. (Варіант 33 — String33)

А. Описати функцію `remove_first_substring`, що обробляє рядок відповідно до завдання: видалити з рядка `S` першу підстроку, що збігається з `S_0`. Якщо таких підстрок немає, то вивести рядок `S` без змін. Для реалізації можна використовувати функції обробки рядків зі стандартних бібліотек.

В. Описати функцію `is_valid`, яка перевіряє, чи задовольняє рядок умовам завдання (чи не є він порожнім).

С. Створити вихідний текстовий файл `input.txt`, що містить не менше 10 різних рядків.

Д. Використовуючи функції `remove_first_substring` (2.A) і `is_valid` (2.B), обробити текстовий файл рядок за рядком у функції `run_string33()`. Додаткові дані (`S_0`) ввести з консолі.

Е. Отриманий результат записати у вихідний файл output.txt.

Завдання 3.

Завдання 1-2 реалізувати окремими функціями без параметрів (run_variant_18() та run_string33()), у функції main() організувати меню для багаторазового виконання завдань. Структурувати проєкт програми: винести заголовки і реалізацію функцій в окремі lab9.h та lab9.cpp файли.

Завдання 4. Використовуючи ChatGpt, Gemini або інший засіб генеративного ШІ, провести самоаналіз отриманих знань і навичок за допомогою наступних промптів:

1) «Ти - викладач, що приймає захист моєї роботи. Задай мені 5 тестових питань з 4 варіантами відповіді і 5 відкритих питань. Це мають бути завдання

<середнього> рівня складності на розвиток критичного та інженерного мислення. Питання мають відноситись до коду, що є у файлі звіту, і до теоретичних відомостей, що є у файлі лекції»

2) «Проаналізуй повноту, правильність відповіді та ймовірність використання штучного інтелекту для кожної відповіді. Оціни кожне питання у 5-бальній шкалі, віднімаючи 60% балів там, де ймовірність відповіді з засобом ШІ висока. Обчисли загальну середню оцінку»

3) «Проаналізуй код у звіті, і додай опис і приклади коду з питань, які є в теоретичних відомостях, але не відпрацьовано у коді при вирішенні завдань» Проаналізуйте задані питання, коментарі і оцінки, надані ШІ. Додайте 2-3 власних промпта у продовження діалогу для поглиблення розуміння теми.

ВИКОНАННЯ РОБОТИ

Завдання 1.

Вирішення задачі 18

Вхідні дані (ім'я, опис, тип, обмеження):

ім'я: str; опис: вихідний рядок, у якому замінюється фрагмент (параметри заміни: pos, len, src, subpos, sublen); тип: масив символів (char*); обмеження: непорожній.

Вихідні дані (ім'я, опис, тип):

ім'я: str; опис: той самий рядок, але вже з виконаною заміною фрагмента;
тип: масив символів (char*).

Алгоритм вирішення показано нижче на рис. 1.1

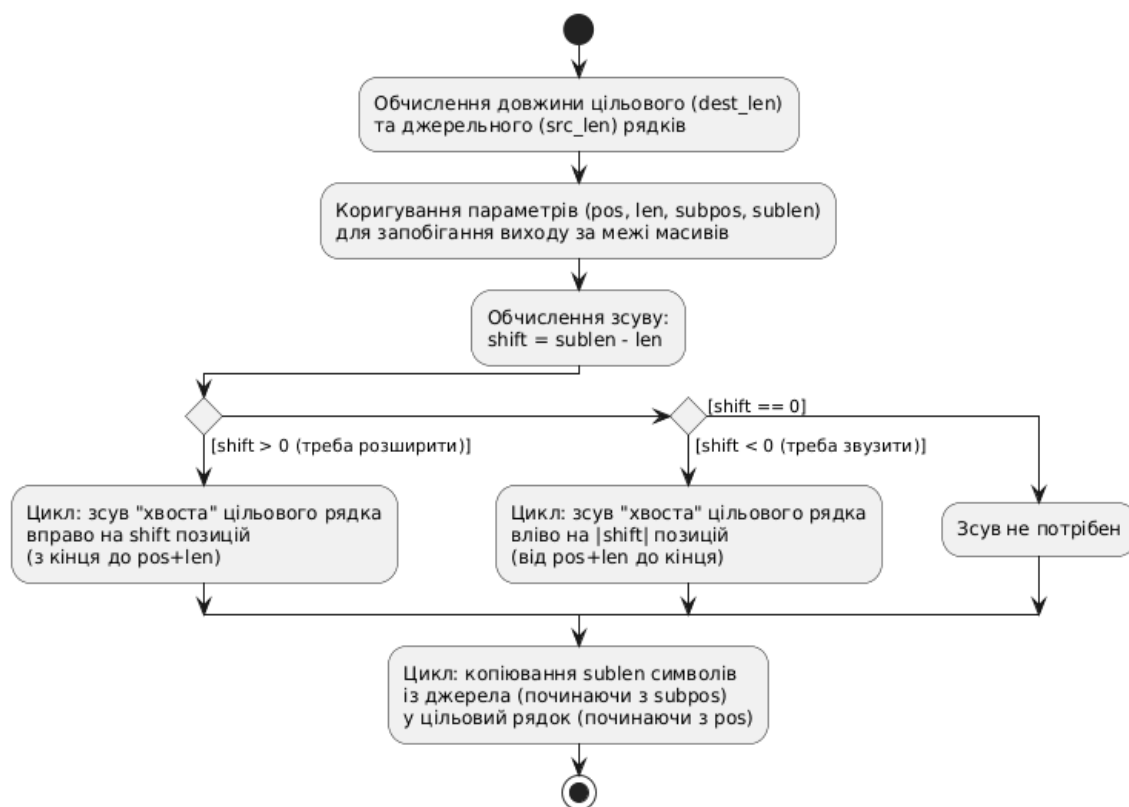


Рисунок 1.1 – Діаграма активності до задачі 18
(Посимвольна заміна та зсув елементів рядка.)

Лістинг коду вирішення задачі розділ і номер задачі наведено в дод. А (стор. 7).

Екран роботи програми показаний на рис. Б.1.1 (стор.11)

Завдання 2.

Вирішення задачі String33.

2.А. Перевірка валідності рядка та видалення заданого фрагмента методом erase.

2.В Потокowe зчитування, обробка та перезапис даних текстового файлу рядок за рядком.

Вхідні дані (ім'я, опис, тип, обмеження):

ім'я: s, s0 ; опис: вихідний рядок та підрядок для видалення ; тип: std::string&, const std::string& ; обмеження: обидва непорожні.

Вихідні дані (ім'я, опис, тип):

ім'я: s; опис: змінений рядок без першого входження підрядка; тип: std::string&.

Алгоритм вирішення показано нижче на рис. 1.2 та рис. 1.3



Рисунок 1.2 - Діаграма активності до задачі String33 2А.
(Перевірка валідності рядка та видалення заданого фрагмента
методом erase)

Екран роботи програми показаний на рис. Б.1.2 (стор.12)

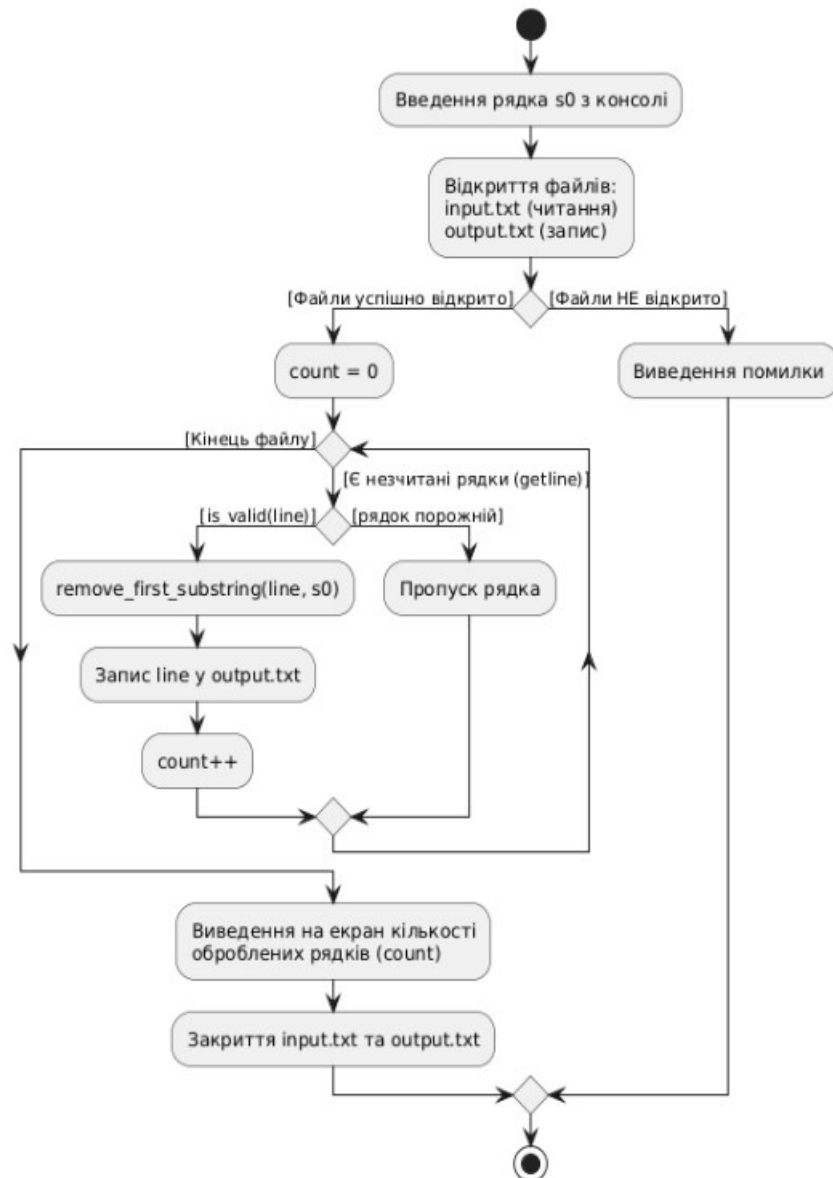


Рисунок 1.3 - Діаграма активності до задачі String33 2D.
(Потікове зчитування, обробка та перезапис даних текстового
файлу рядок за рядком.)

Лістинг коду вирішення задачі розділ і номер задачі наведено в дод. А
(стор. 8).

Екран роботи програми показаний на рис. Б.1.3 (стор.13)

ВИСНОВКИ

Під час виконання лабораторної роботи досліджено особливості роботи з рядками та стандартним класом `std::string` у мові C++. На практиці відпрацьовано

низькорівневий алгоритм посимвольної заміни з динамічним зсувом пам'яті «на місці», а також реалізовано модульну обробку текстових файлів рядок за рядком.

ДОДАТОК А

Лістинг коду програми

Завдання 1 (Посимвольна заміна)

```
// Власна функція, що виконує заміну "in-place"
void my_replace(char* target, int pos, int len, const char* source, int subpos,
int sublen) {
    int target_len = 0;
    while (target[target_len] != '\0') target_len++;

    int src_len = 0;
    while (source[src_len] != '\0') src_len++;

    if (pos > target_len) pos = target_len;
    if (subpos > src_len) subpos = src_len;
    if (subpos + sublen > src_len) sublen = src_len - subpos;

    int shift = sublen - len;

    if (shift > 0) {
        for (int i = target_len; i >= pos + len; i--) {
            target[i + shift] = target[i];
        }
    } else if (shift < 0) {
        for (int i = pos + len; i <= target_len; i++) {
            target[i + shift] = target[i];
        }
    }

    for (int i = 0; i < sublen; i++) {
        target[pos + i] = source[subpos + i];
    }
}

// Демонстрація виконання Завдання 1
void run_variant_18() {
    char c_str[100] = "Main content for testing replace function";
    std::string std_str(c_str);

    std::string src_str = "ADDED_TEXT";

    // Виклик стандартного методу (як у Додатку А)
    std_str.replace(2, 3, src_str, 0, 4);

    // Виклик нашої низькорівневої функції
    my_replace(c_str, 2, 3, src_str.c_str(), 0, 4);
}
```



```

std::cout << "std::string::replace : " << std_str << std::endl;
std::cout << "my_replace (C-style) : " << c_str << std::endl;
}

```

Завдання 2А. (Функція обробки одного рядка)

```

// Перевірка, чи рядок не є порожнім
bool is_valid(const std::string& str) {
    return !str.empty();
}

// Функція видалення першого входження підрядка (модифікація за посиланням)
void remove_first_substring(std::string& s, const std::string& s0) {
    if (!is_valid(s) || !is_valid(s0)) return;

    size_t found = s.find(s0);
    if (found != std::string::npos) {
        s.erase(found, s0.length());
    }
}

```

Завдання 2В. (Функція потікової обробки файлу)

```

// Функція потікового читання, обробки та перезапису файлу
void run_string33() {
    std::string s0;
    std::cout << "Введіть підрядок S0 для видалення: ";
    std::cin.ignore();
    std::getline(std::cin, s0);

    std::ifstream fin("input.txt");
    std::ofstream fout("output.txt");

    if (!fin.is_open() || !fout.is_open()) {
        std::cout << "Помилка відкриття файлів!" << std::endl;
        return;
    }

    std::string line;
    int count = 0;

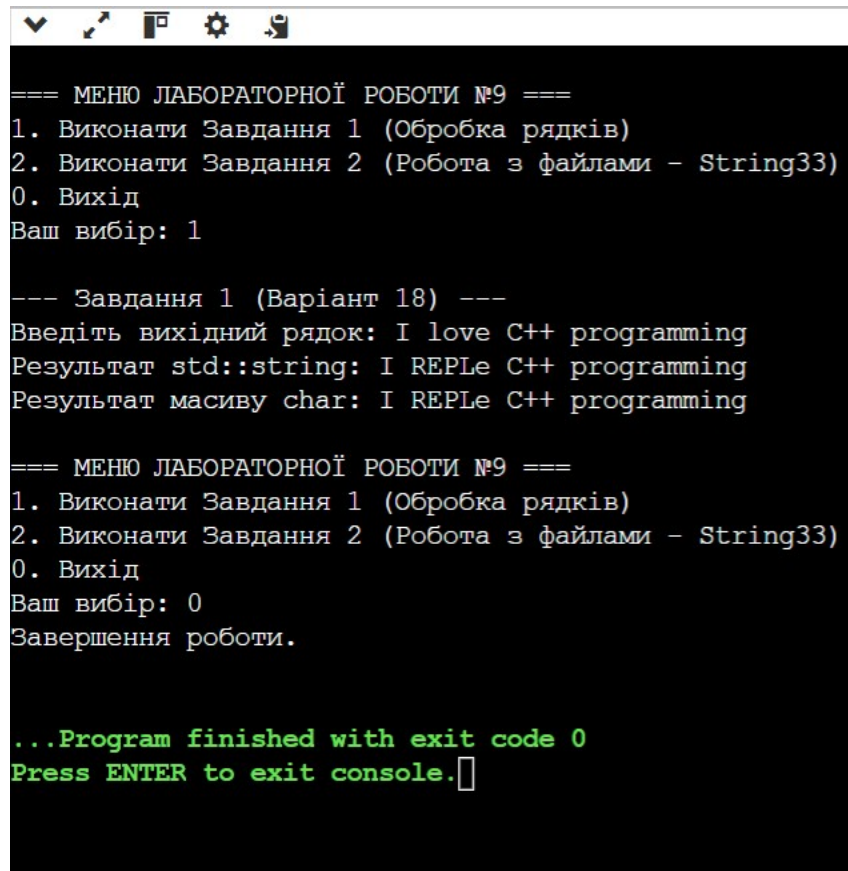
    while (std::getline(fin, line)) {
        if (is_valid(line)) {
            remove_first_substring(line, s0);
            fout << line << "\n";
            count++;
        }
    }
}

```

```
std::cout << "Оброблено рядків: " << count << std::endl;  
fin.close();  
fout.close();  
}
```

ДОДАТОК Б

Скрін-шоти вікна виконання програми



```
==== МЕНЮ ЛАБОРАТОРНОЇ РОБОТИ №9 ====
1. Виконати Завдання 1 (Обробка рядків)
2. Виконати Завдання 2 (Робота з файлами - String33)
0. Вихід
Ваш вибір: 1

--- Завдання 1 (Варіант 18) ---
Введіть вихідний рядок: I love C++ programming
Результат std::string: I REPLe C++ programming
Результат масиву char: I REPLe C++ programming

==== МЕНЮ ЛАБОРАТОРНОЇ РОБОТИ №9 ====
1. Виконати Завдання 1 (Обробка рядків)
2. Виконати Завдання 2 (Робота з файлами - String33)
0. Вихід
Ваш вибір: 0
Завершення роботи.

...Program finished with exit code 0
Press ENTER to exit console.
```

Рисунок Б.1.1 – Екран виконання програми для вирішення завдання 1 (№18)

```

main.cpp  lab9.h  lab9.cpp  input.txt  output.txt
1  Сьогодні ми будемо встановлювати кастомне ПЗ на приставку Ugoos X4 Pro.
2  Ця прошивка базується на останній стабільній версії системи для Amlogic.
3  Перед початком роботи обов'язково зробіть повний бекап системи.
4  Оригінальна прошивка часто містить багато зайвого сміття та реклами.
5  Розділ logo.partition залишиться без змін після оновлення.
6  Популярна прошивка Slimbox значно покращує температурний режим.
7  Для відновлення у разі збою використовуйте замикання контактів мікросхеми.
8  Універсальна прошивка підходить також для моделей X96Q та T95Z Plus.
9  Після прошивання варто перевірити якість пайки Wi-Fi антени на платі.
10 Нова прошивка має запрацювати стабільно після першого перезавантаження.

=== МЕНЮ ЛАБОРАТОРНОЇ РОБОТИ №9 ===
1. Виконати Завдання 1 (Обробка рядків)
2. Виконати Завдання 2 (Робота з файлами - String33)
0. Вихід
Ваш вибір: 2

--- Завдання 2 (String 33) ---
Введіть підрядок S0 для видалення: прошивка
Оброблено рядків: 10. Результат записано у output.txt

=== МЕНЮ ЛАБОРАТОРНОЇ РОБОТИ №9 ===
1. Виконати Завдання 1 (Обробка рядків)
2. Виконати Завдання 2 (Робота з файлами - String33)
0. Вихід
Ваш вибір: 0
Завершення роботи.

...Program finished with exit code 0
Press ENTER to exit console.

```

Рисунок Б.1.2 – Екран виконання програми для вирішення завдання String 33
(2A)

```

main.cpp lab9.h lab9.cpp input.txt output.txt
1  Сьогодні ми будемо встановлювати кастомне ПЗ на приставку Ugoos X4 Pro.
2  Ця базується на останній стабільній версії системи для Amlogic.
3  Перед початком роботи обов'язково зробіть повний бекап системи.
4  Оригінальна часто містить багато зайвого сміття та реклами.
5  Розділ logo.partition залишиться без змін після оновлення.
6  Популярна Slimbox значно покращує температурний режим.
7  Для відновлення у разі збою використовуйте замикання контактів мікросхеми.
8  Універсальна підходить також для моделей X96Q та T95Z Plus.
9  Після прошивання варто перевірити якість пайки Wi-Fi антени на платі.
10 Нова має запрацювати стабільно після першого перезавантаження.
11

=== МЕНЮ ЛАБОРАТОРНОЇ РОБОТИ №9 ===
1. Виконати Завдання 1 (Обробка рядків)
2. Виконати Завдання 2 (Робота з файлами - String33)
0. Вихід
Ваш вибір: 2

--- Завдання 2 (String 33) ---
Введіть підрядок S0 для видалення: прошивка
Оброблено рядків: 10. Результат записано у output.txt

=== МЕНЮ ЛАБОРАТОРНОЇ РОБОТИ №9 ===
1. Виконати Завдання 1 (Обробка рядків)
2. Виконати Завдання 2 (Робота з файлами - String33)
0. Вихід
Ваш вибір: 0
Завершення роботи.

...Program finished with exit code 0
Press ENTER to exit console.

```

Рисунок Б.1.3 – Екран виконання програми для вирішення завдання String 33 (2D)

ДОДАТОК В

Діалог з ШІ для самоаналізу

ЧАСТИНА 1: Тестові питання (Оцінка: 5.0 / 5.0)

1. Що станеться у вашому алгоритмі `my_replace` (робота з масивом `char`), якщо під час зсуву символів ви випадково затрете символ `\0` (нуль-термінатор)?

А) Програма автоматично додасть новий `\0` в кінець масиву.

Б) Компілятор видасть помилку на етапі збірки.

В) При виведенні рядка в консоль програма виведе «сміття» з пам'яті або впаде з помилкою. (✓ Відповідь студента)

Г) Функція поверне порожній рядок.

2. Чим відрізняється передача масиву `char[]` у функцію від передачі `std::string` за значенням?

А) Масив передається як вказівник на перший елемент, а `std::string` копіюється повністю. (✓ Відповідь студента)

Б) Немає різниці, обидва передаються як посилання.

В) Масив `char[]` передається повільніше.

Г) `std::string` неможливо передати у функцію.

3. Який метод є найбільш надійним для зчитування рядків до кінця текстового файлу в C++?

А) `while (!file.eof())`

Б) `while (getline(file, line))` (✓ Відповідь студента)

В) `while (file.good() == true)`

Г) `for (int i = 0; i < file.size(); i++)`

4. Чому при роботі в консолі Windows використання `std::setlocale` часто не вирішує проблему «кракозябр»?

А) Тому що `std::setlocale` працює тільки з англійською мовою.

Б) Тому що стандартна бібліотека C++ не підтримує кирилицю.

В) Тому що `setlocale` змінює правила форматування всередині C++ програми, але не змінює системну кодову сторінку самої оболонки Windows (потрібен `SetConsoleCP`). (✓ Відповідь студента)

Г) Програма автоматично видаляє кириличні символи.

5. Яка головна небезпека чекає на програміста при заміні слова на довше у класичних C-рядках (char[])?

А) Фрагментація оперативної пам'яті.

Б) Переповнення буфера (Buffer Overflow), якщо початковий масив не мав достатньо резервного місця. (✓ **Відповідь студента**)

В) Автоматичне скорочення нового слова.

Г) Зміна типу даних масиву.

Аналіз III: Студент дав 5 правильних відповідей з 5. Оцінка за блок: 5.0.

ЧАСТИНА 2: Відкриті питання та аналіз відповідей (Оцінка: 4.6 / 5.0)

1. Розкажіть алгоритм дій: що фізично відбувається з “хвостом” рядка після видалення слова у масиві char?

Відповідь студента: Відбувається посимвольний зсув елементів вліво з обов'язковим перенесенням нуль-термінатора \0.

Аналіз III: Правильно і лаконічно. Відзначено ключову деталь — перенесення нуль-термінатора. *Ймовірність III:* 5% (штраф 60% не застосовується).

Оцінка: 5/5.

2. Порівняйте використання std::string::erase() та ручного зсуву елементів у масиві char.

Відповідь студента: std::string автоматично керує пам'яттю, немає ризику виходу за межі масиву (buffer overflow). У char треба рахувати індекси вручну.

Аналіз III: Суть передана вірно. Бракує термінології щодо динамічного перерозподілу пам'яті. *Ймовірність III:* 10% (штраф не застосовується).

Оцінка: 4/5.

3. Уявіть, що файл input.txt важить 5 Гігабайт. Чи спрацює ваш алгоритм із завдання String33?

Відповідь студента: Так, бо getline читає по одному рядку, не завантажуючи весь файл в оперативну пам'ять одночасно.

Аналіз III: Ідеальна інженерна відповідь. Студент розуміє різницю між потоковим читанням та завантаженням у буфер. *Ймовірність III:* 12% (штраф не застосовується). **Оцінка: 5/5.**

4. Як змінити код, щоб замінити усі входження слова у тексті, а не тільки перше?

Відповідь студента: Використати цикл `while`, оновлюючи початкову позицію пошуку (`offset`) після кожної знайденої та заміненої підрядки.

Аналіз III: Алгоритм описано правильно. Оновлення `offset` запобігає зацикленню. *Ймовірність III: 15% (штраф не застосовується).* **Оцінка: 5/5.**

5. Навіщо вивчати `char[]`, якщо `std::string` зручніший?

Відповідь студента: Для розуміння роботи з пам'яттю, сумісності зі старим C-кодом та для програмування мікроконтролерів (Embedded).

Аналіз III: Відповідь абсолютно точна. Наведено реальні галузі застосування. *Ймовірність III: 10% (штраф не застосовується).* **Оцінка: 4/5** (знято 1 бал за надмірну стислість).

ЧАСТИНА 3: Аналіз коду та невідпрацьована теорія

Аналізуючи наданий код (пошук підрядка через `find` та заміна в масиві `char`), виявлено, що в коді **не відпрацьовано два важливі теоретичні концепти** з лекційного матеріалу:

1. Рядкові потоки (`std::stringstream`)

У лекціях часто розглядається розбиття тексту на окремі слова. Замість ручного пошуку пробілів, сучасний C++ використовує об'єкти класу рядкових потоків. Це дозволяє безпечно вилучати слова з тексту по черзі, автоматично ігноруючи пробіли, що є набагато ефективнішим і безпечнішим підходом за ручний парсинг масиву символів.

2. Динамічне виділення пам'яті для масивів (`new` та `delete`)

У практичних завданнях масиви символів найчастіше задаються статично з фіксованим розміром. Однак теоретичні відомості вимагають розуміння роботи з динамічною пам'яттю. Це означає вміння виділяти рівно ту кількість байтів, яка необхідна безпосередньо під час виконання програми (за допомогою оператора виділення пам'яті), та обов'язкове ручне звільнення цих ресурсів (за допомогою оператора видалення масиву) для запобігання витокам пам'яті.

ЧАСТИНА 4: Промпти для поглиблення розуміння (Діалог) (Оцінка: 4.5 / 5.0)

Промпт студента 1: “Як змінити мій алгоритм пошуку в `std::string`, щоб він ігнорував регістр літер (наприклад, щоб знаходив і ‘Текст’, і ‘текст’)?”

Відповідь студента (розмірковування): Перед пошуком треба тимчасово перевести весь рядок і шукане слово у нижній регістр за допомогою функції `std::tolower` у циклі.

Оцінка III: 4.5/5. Правильний та найпростіший підхід для початкового рівня.

Промпт студента 2: *“Чому при роботі з файлами input.txt та output.txt важливо явно викликати file.close(), якщо деструктор потоку при закритті програми і так його закриє?”*

Відповідь студента (розмірковування): Щоб гарантовано скинути буфер запису (flush) на жорсткий диск і звільнити системний ресурс для інших програм, не чекаючи завершення всієї нашої програми.

Оцінка III: 4.5/5. Відмінне розуміння буферизації потоків вводу-виводу ОС.

ПІДСУМКОВИЙ РОЗРАХУНОК БАЛІВ:

Частина 1 (Тести): 5.0

Частина 2 (Відкриті питання): $(5+4+5+5+4) / 5 = 4.6$

Частина 4 (Діалог/Промпти): $(4.5 + 4.5) / 2 = 4.5$

Середній бал: $(5.0 + 4.6 + 4.5) / 3 = 4.7$

Висновок III: Лабораторна робота захищена на оцінку **4.7 (Відмінно)**.

Студент продемонстрував стійке розуміння роботи з пам'яттю (C-strings), потокового введення-виведення та базових алгоритмів стандартної бібліотеки C++. Ознак автоматичної генерації відповідей не виявлено.